Prepare answers before the exercise class. In class you will present them.

2.1 Does the following statement satisfy the synchronous assumption?

"On my server, no request ever takes more than 1 week to be processed."

Sample solution.

This question is somewhat inaccurate (exam questions will most likely be less ambiguous).

- *If interpreted as "is the process behaviour synchronous": 1 week is a bounded known value, so the answer is yes.*
- *If interpreted as "is the distributed system synchronous": No statement about the network is made (and that usually is the more difficult part), and a synchronous behaviour of a node is not sufficient to classify the whole distributed system as synchronous, so the answer is no.*

2.2 Models in distributed systems

a) T F (T=true/F=false)

☐ ☐ A model is accurate only if it is tractable

☐ ☐ Models of distributed systems make it possible to predict the feasibility and costs of algorithms or protocols

☐ ☐ In an asynchronous system you make the assumption that for most messages the time between sending and receiving the message is infinite

☐ ☐ Using a Byzantine fault model instead of a crash fault model has only advantages

☐ ☐ Algorithms designed for an asynchronous system model can be used without modification in a synchronous system.

b) Can you describe a case (some specific assumptions) in which the crash-recovery and the fail-silent models are practically identical?

Sample solution.

***Comments**: F/T*/F/F/T*

- *Accurate does not necessarily imply that it is tractable*
- *Feasibility and costs are two important aspects you might want to analyse/predict based on some model (however, you might argue that sometimes you'll have a model that does not allow you to answer all possible questions regarding feasibility and/or cost)*
- *In asynchronous systems we don't make any assumptions neither for communication delays nor for computational time*
- *Providing fault tolerance in a Byzantine crash model requires more resources and most of the time a more complex communication scheme, so it definitely has not only advantages*
- *Those algorithms will work without any adaptations (the reverse does not hold!)*

2.3 Properties of logical clocks

The relation "a happen causally before b" is written as $a \rightarrow b$. Two events, a and b, have known logical (Lamport) time stamps $C(a)$ and $C(b)$, such that $C(a) < C(b)$.

a) What can you say about the correctness of the following statements (certainly true/certainly false/undecidable)?

- $a \rightarrow b$
- $b \rightarrow a$
- a happend before b in real time
- b happend before a in real time
- a and b are concurrent (i.e., neither $a \rightarrow b$ nor $b \rightarrow a$ is true)

Sample solution.

Lamport's logical clocks map each event in processes to a counter value (integer) and can be written as $C(event)$. *They can be implemented using counters with no relation to realtime. These counters have to satisfy one condition. If* a *and* b *are two events in a system and* a *happened-before* b *(*$a \rightarrow b$*) then* $C(a)$ *has to be less than* $C(b)$.

- *If two* a *and* b *events happen in the same process* i, *the process* i *increments the clock* C_i *between any two successive events so* $C_i(a) < C_i(b)$.
- *If event* a *is sending the message* m *event from process* i *to process* j, *the process* i *timestamps the message with its own logical clock* $t_m = C_i(a)$. *When process* j *receives the message* m, *it sets its clock to* $C_j(b)$ *greater than its present value and greater than* t_m, *i.e.* $C_j' := max(C_j, t_m) + 1$

It is important to notice that while $a \rightarrow b \Rightarrow C(a) < C(b)$, $a \rightarrow b \nLeftrightarrow C(a) < C(b)$.

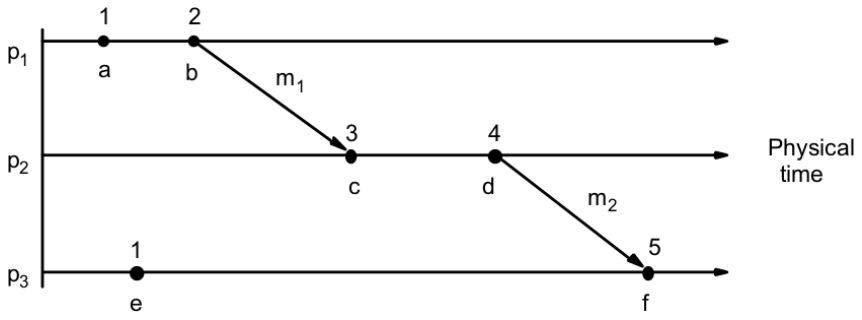


Fig 1. Logical clocks

1. $a \rightarrow b$ – *undecidable*
2. $b \rightarrow a$, *or* b *happened before* a, *which if it were true, it would result in* $C(a) > C(b)$, *but in our hypothesis we have that* $C(a) < C(b)$ – *certainly false.*
3. a *happened before* b *in real time – undecidable*
4. b *happened before* a *in real time – undecidable*
5. a *and* b *are concurrent – undecidable. The concurrency relation can be defined as* $a \parallel b$ *if and only if* $\neg(a \rightarrow b) \wedge \neg(b \rightarrow a)$. *Although we rule out that* $\neg(b \rightarrow a)$, *we can't assert that* $\neg(a \rightarrow b)$, *it may be either true or false – undecidable*

b) What can you say about these statements, if you replace the time stamps with values of a vector clock $V(a)$ and $V(b)$, and again assume $V(a) < V(b)$?

Sample solution.

From the fact $C_i(a) < C_j(b)$ no one can conclude that a happened-before b. To overcome this shortcoming the concept of vector clocks was developed. A vector clock is very similar to the one that Lamport used but it includes a separate counter value for each process. This vector is used to timestamp each message exchanged in the system.

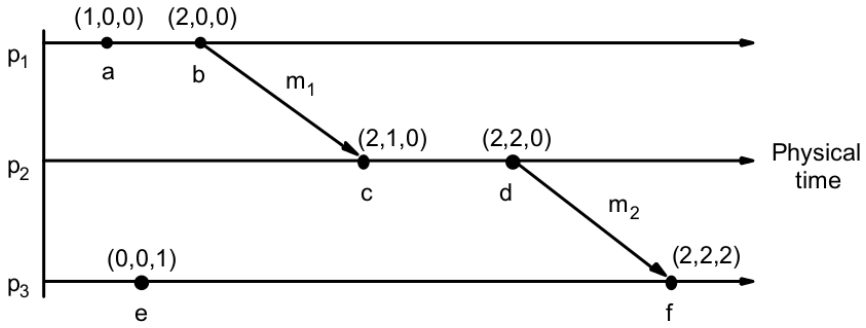


Fig 2. Vector clocks

As it stands, using vector clocks we can write that: $a \rightarrow b \Leftrightarrow V(a) < V(b)$.

1. $a \rightarrow b$ – *certainly true*
2. $b \rightarrow a$ – *certainly false*
3. a *happened before* b *in real time – certainly true*
4. b *happened before* a *in real time – certainly false*
5. a *and* b *are concurrent – certainly false*

Note that even if the time stamps have not relation to real time, we can conclude from the timestamps that two events have a causal relation, and obviously the "causing" event happens before the "cause" event in realtime.

Only if neither V(a)<V(b) nor V(b)<V(a) holds, then we don't known which of the two events a,b, happended earlier in real time.

2.4 Logical clocks and fault tolerance

What effect can a crash of a single process have on Lamport logical clocks with respect to the basic ordering property of the algorithm? (i.e., is it possible that due to a crash the order property $a \rightarrow b \Longrightarrow C(a) < C(b)$ is violated?)

Sample solution.

If we assume a simple crash model (fail-stop, fail-silent, even crash-recovery), nothing bad can happen. If a node assigns a time stamp to an event or to an message, this node is still working correctly at this point in time, and the locally generated time stamp will consider all causally preceding events.

(Note that the situation is completely different in a Byzantine fault model.)

2.5 Total order with vector clocks

Lamport logical clocks create a partial order. This means that multiple events can have the same time stamp, which means that there is no unique order for these events. As seen in the lecture, you can extend this partial order to a **total order** by using tuples of Lamport clock and node ID (C_{P_k}, k), and defining a predicate "$<$" on two tuples in an appropriate way.

Now assume you have vector clocks instead of Lamport clocks. They also define a partial order. Explain how to define a **total order** on the basis of vector clocks!

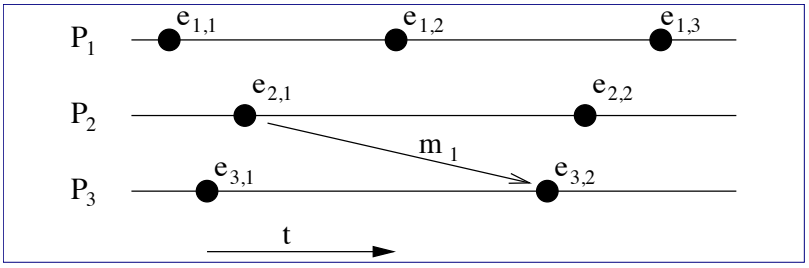
Sample solution.

One possibility (others exist): For vector clocks we can use $\sum V(event)$ *(as the sum of all elements of the vector) and order elements with equal sums by using the process id:*

$$(V_i, i) < (V_k, k) \Leftrightarrow \sum V_i < \sum V_k \vee (\sum V_i = \sum V_k \wedge i < k) \quad (1)$$

2.6 Logical clocks – example

Consider this set of events and interactions between nodes:



a) Assign Lamport timestamps to each event, describe an algorithm for deriving a total order from Lamport timestamps (see also lecture), and sort all events according to this order.

b) Assign vector timestamps to each event, describe an algorithm for deriving a total order from these vector timestamps (see also 1.9), and sort all events according to this order.